

Vector Concepts in MATLAB

MATLAB Fundamentals

A teaching note on vector creation, operations, indexing, and practice exercises

Prepared for students learning introductory MATLAB programming

March 19, 2026

Contents

1	Introduction	2
2	Vector Definitions	2
2.1	Row and Column Vectors	2
2.2	Examples in MATLAB	2
3	Creating Vectors in Different Ways	3
3.1	Manual Entry	3
3.2	Using the Colon Operator: <code>start:finish</code>	3
3.3	Using the Colon Operator: <code>start:step:finish</code>	3
3.4	Using <code>linspace</code>	4
3.5	Colon Operator vs <code>linspace</code>	4
4	Mathematical Operations on Vectors	4
4.1	Addition and Subtraction	4
4.2	Scalar Multiplication	5
4.3	Matrix Multiplication (Inner Product)	5
4.4	Element-wise Operations	5
5	Slicing and Indexing	6
5.1	Explanation of Indexing	6
5.2	More Examples	6
6	Norm and Magnitude	6
7	Worked Examples	7
7.1	Example 1: Creating a Sequence	7
7.2	Example 2: Creating 8 Equally Spaced Values	7
7.3	Example 3: Selecting Part of a Vector	7
7.4	Example 4: Element-wise Squaring	7
8	Common Mistakes Students Should Avoid	7
9	Self Exercises	8
10	Challenge Tasks	9
11	Conclusion	9

1 Introduction

Vectors are among the most important data structures in MATLAB. In fact, MATLAB is designed to work very efficiently with arrays, matrices, and vectors. A vector may be viewed as a one-dimensional list of values arranged either horizontally or vertically.

A vector is essentially:

- a **row vector** of size $1 \times n$, or
- a **column vector** of size $m \times 1$.

Understanding vectors is important because many tasks in engineering, science, and computing involve storing sequences of numbers, performing calculations on data, and selecting subsets of values.

2 Vector Definitions

2.1 Row and Column Vectors

A **row vector** is written with elements separated by spaces or commas:

$$v = [v_1 \ v_2 \ v_3 \ \dots \ v_n]$$

A **column vector** is written with elements separated by semicolons:

$$w = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_m \end{bmatrix}$$

2.2 Examples in MATLAB

Listing 1: Creating Row and Column Vectors

```

1 a = [1, 2, 3];           % Row vector
2 b = [4; 5; 6];          % Column vector
3 b_row = b';              % Transpose column vector to row vector

```

Explanation:

- **a** contains three elements in one row.
- **b** contains three elements in one column.
- The apostrophe operator `'` transposes the vector.

3 Creating Vectors in Different Ways

3.1 Manual Entry

The simplest way to create a vector is to type each value directly.

Listing 2: Manual Vector Entry

```
1 x = [2, 4, 6, 8, 10];
2 y = [3; 6; 9; 12];
```

This approach is useful when the vector has only a few values or when the values are not evenly spaced.

3.2 Using the Colon Operator: **start:finish**

MATLAB allows vectors to be created using the colon operator. When the step size is omitted, MATLAB assumes a step of 1.

Listing 3: Creating Vectors with start:finish

```
1 v = 1:5;      % Produces [1 2 3 4 5]
```

This means:

$$v = [1 \ 2 \ 3 \ 4 \ 5]$$

3.3 Using the Colon Operator: **start:step:finish**

A more general form is:

start:step:finish

This creates a vector beginning at **start**, increasing or decreasing by **step**, and stopping at or before **finish**.

Listing 4: Creating Vectors with start:step:finish

```
1 a = 2:2:10;      % [2 4 6 8 10]
2 b = 1:0.5:3;     % [1 1.5 2 2.5 3]
3 c = 10:-2:2;     % [10 8 6 4 2]
```

Important notes:

- If the step is positive, the values increase.
- If the step is negative, the values decrease.
- MATLAB only includes values that fit the step pattern.

For example:

```
1 d = 1:2:8;      % [1 3 5 7]
```

The value 8 is not included because starting from 1 and moving in steps of 2 gives:

$$1, 3, 5, 7, 9$$

Since 9 goes beyond 8, MATLAB stops at 7.

3.4 Using linspace

Another useful MATLAB command is `linspace`. Its general form is:

```
linspace(start, finish, number_of_points)
```

It creates a vector with a specified number of equally spaced points between the start and finish values.

Listing 5: Creating Vectors with linspace

```
1 x = linspace(0, 10, 6);      % [0 2 4 6 8 10]
2 y = linspace(1, 5, 9);      % 9 equally spaced points from 1 to 5
```

Why use linspace?

- It is useful when you want an exact number of values.
- It is especially common in plotting graphs.
- Unlike the colon operator, you specify the number of points instead of the step size.

3.5 Colon Operator vs linspace

Method	You specify	Best for
<code>start:step:finish</code>	start, step, finish	known interval size
<code>linspace(start,finish,n)</code>	start, finish, number of points	known number of values

Example:

```
1 a = 0:2:10;      % step size is 2
2 b = linspace(0,10,6); % 6 equally spaced points
```

Both produce:

```
[0 2 4 6 8 10]
```

4 Mathematical Operations on Vectors

4.1 Addition and Subtraction

Vectors of the same size can be added or subtracted element by element.

Listing 6: Vector Addition and Subtraction

```
1 a = [1, 2, 3];
2 b = [4, 5, 6];
3
4 sum_vec = a + b;      % [5 7 9]
5 diff_vec = b - a;     % [3 3 3]
```

4.2 Scalar Multiplication

A scalar is a single number. Multiplying a vector by a scalar multiplies each element by that number.

Listing 7: Scalar Multiplication

```
1 v = [2, 4, 6];
2 result = 3 * v;           % [6 12 18]
```

4.3 Matrix Multiplication (Inner Product)

To compute the dot product of two vectors, the dimensions must match correctly.

For vectors

$$a = [a_1 \ a_2 \ \dots \ a_n], \quad b = [b_1 \ b_2 \ \dots \ b_n]$$

the inner product is:

$$a \cdot b = \sum_{i=1}^n a_i b_i$$

In MATLAB, if both are row vectors, one must be transposed:

Listing 8: Dot Product / Inner Product

```
1 a = [1, 2, 3];
2 b = [4, 5, 6];
3
4 dot_product = a * b';      % 32
```

Since:

$$1(4) + 2(5) + 3(6) = 4 + 10 + 18 = 32$$

4.4 Element-wise Operations

In MATLAB, element-wise calculations use the dot operator.

Listing 9: Element-wise Multiplication and Powers

```
1 x = [1, 2, 3];
2 y = [4, 5, 6];
3
4 z_dot = x .* y;           % [4 10 18]
5 z_pow = x .^ 2;           % [1 4 9]
6 z_div = y ./ x;           % [4 2.5 2]
```

Explanation:

- `.*` multiplies corresponding elements.
- `./` divides corresponding elements.
- `.^` raises each element to a power.

5 Slicing and Indexing

MATLAB uses **1-based indexing**. This means the first element is at position 1, not 0.

Listing 10: Indexing and Slicing

```

1 v = [10, 20, 30, 40, 50];
2
3 val1 = v(1);           % First element: 10
4 val3 = v(3);           % Third element: 30
5 sub = v(2:4);          % [20 30 40]
6 last = v(end);         % Last element: 50

```

5.1 Explanation of Indexing

If

$$v = [10 \ 20 \ 30 \ 40 \ 50]$$

then:

- `v(1)` gives 10
- `v(2)` gives 20
- `v(2:4)` gives the subvector `[20 30 40]`
- `v(end)` gives the last value

5.2 More Examples

Listing 11: More Indexing Examples

```

1 p = [5, 10, 15, 20, 25, 30];
2
3 first_three = p(1:3);      % [5 10 15]
4 middle = p(3:5);          % [15 20 25]
5 all_but_last = p(1:end-1); % [5 10 15 20 25]

```

6 Norm and Magnitude

The Euclidean norm of a vector v is:

$$\|v\| = \sqrt{\sum_{i=1}^n |v_i|^2}$$

In MATLAB, this is computed using:

Listing 12: Computing the Norm of a Vector

```

1 v = [3, 4];
2 magnitude = norm(v);    % 5

```

Since:

$$\|v\| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

The norm is useful for measuring the length or magnitude of a vector.

7 Worked Examples

7.1 Example 1: Creating a Sequence

Create a vector from 0 to 20 in steps of 5.

```
1 a = 0:5:20;
```

Output:

[0 5 10 15 20]

7.2 Example 2: Creating 8 Equally Spaced Values

```
1 b = linspace(2, 18, 8);
```

This creates 8 equally spaced values between 2 and 18.

7.3 Example 3: Selecting Part of a Vector

```
1 v = [2, 4, 6, 8, 10, 12];  
2 part = v(2:5);
```

Output:

[4 6 8 10]

7.4 Example 4: Element-wise Squaring

```
1 x = [1, 3, 5, 7];  
2 y = x.^2;
```

Output:

[1 9 25 49]

8 Common Mistakes Students Should Avoid

1. Forgetting that MATLAB starts indexing at 1.
2. Using `*` instead of `.*` for element-wise multiplication.
3. Mixing row vectors and column vectors without checking dimensions.
4. Assuming `start:step:finish` always includes the finish value.
5. Confusing `linspace` with the colon operator.

9 Self Exercises

Exercise Set A: Basic Practice

1. Create the row vector:

$$[2 \ 4 \ 6 \ 8 \ 10]$$

2. Create the column vector:

$$\begin{bmatrix} 3 \\ 6 \\ 9 \\ 12 \end{bmatrix}$$

3. Use the colon operator to create a vector from 1 to 15.

4. Use `start:step:finish` to create the vector:

$$[5 \ 10 \ 15 \ 20 \ 25]$$

5. Create a descending vector from 20 to 0 in steps of 4.

6. Use `linspace` to generate 5 equally spaced points from 0 to 1.

7. Let `a = [1,2,3]` and `b = [4,5,6]`. Compute:

- (a) `a + b`
- (b) `a .* b`
- (c) `a * b'`

8. For `v = [10,20,30,40,50,60]`, find:

- (a) the first element
- (b) the fourth element
- (c) the subvector from the second to the fifth element
- (d) the last element

9. Compute the norm of `[6, 8]`.

10. Write MATLAB code to square each element of the vector `[2, 4, 6, 8]`.

Exercise Set B: Thinking Questions

1. What is the difference between a row vector and a column vector?
2. When would you prefer `linspace` instead of `start:step:finish`?
3. Why does MATLAB require `.*` for element-wise multiplication?
4. Explain why `1:2:8` does not include 8.
5. What is the importance of the `end` keyword in indexing?

10 Challenge Tasks

1. Create a vector from 0 to 50 in steps of 5, then extract only the last four elements.
2. Generate 11 equally spaced points from -5 to 5 using `linspace`, then square all elements.

3. Let

$$a = [2, 4, 6, 8], \quad b = [1, 3, 5, 7]$$

Find:

- (a) `a + b`
- (b) `a - b`
- (c) `a .* b`
- (d) `a * b'`

4. Create a column vector with values from 10 down to 2 in steps of 2.
5. Write a short MATLAB script that:
 - (a) creates a vector using `linspace`
 - (b) displays the first and last elements
 - (c) computes the norm
 - (d) squares each element

11 Conclusion

Vectors are a core part of MATLAB programming. Students should be comfortable with:

- defining row and column vectors,
- creating vectors manually and automatically,
- using `start:step:finish`,
- using `linspace`,
- performing mathematical operations,
- indexing and slicing vectors, and
- computing vector magnitude using `norm`.

Mastering these basics will make it much easier to work with matrices, graphs, engineering data, simulations, and numerical methods later in MATLAB.